

How a Promise Behaves

States, settling, await, why a try/catch only sees what is awaited inside it, and the three kinds of chaining

October 2026

A Promise is a ticket for a value that arrives later. Calling an async function (or `fetch`, `db.query`, anything from the Promise list) starts the work and hands you the ticket immediately. The function that handed it over is finished; the work continues in the background.

Three states, one transition. A Promise starts *pending* and later *settles* exactly once, into one of two outcomes. It never goes back.

State	Meaning	What <code>await</code> does with it
pending	work not finished	pauses the current function until it settles
fulfilled	finished with a value	gives you the value
rejected	finished with an error	throws that error at the <code>await</code> line

“Settles” means fulfilled OR rejected. A pending Promise has not failed; it has not done anything yet.

`await` is the only thing that opens the ticket. `await p` means: stop this function here, keep its place, let other requests run; when `p` settles, resume: if fulfilled, hand back the value; if rejected, throw the error on this exact line. Without `await`, `p` is just an object you are holding, and nothing you do to it (`p.name`, `p + 1`) touches the value inside. (`.then(f)` is the older way to open it; same idea.)

Being a Promise and being awaited are separate facts. `fetch()` returns a Promise whether or not anyone waits. Nothing forces a caller to `await`; it can walk away, or hand the Promise onward with `return`. The Promise still runs and still settles; the question is whether anyone is listening when it does.

`catch` runs only if an error is thrown while execution is still inside the `try` block. A rejection is not a throw. It becomes a throw at the moment something `await`s it. So the `try` covers a Promise only if the `await` on that Promise is inside the block, before the block ends. If the block exits first, that `try` is over; it does not wait around.

no <code>await</code>	with <code>await</code>
t=0 enter try	t=0 enter try
t=0 <code>fetch()</code> starts; returns pending Promise	t=0 <code>fetch()</code> starts; returns pending Promise
t=0 <code>return</code> -> exit try; nothing thrown;	t=0 <code>await</code> -> PAUSE here, still inside try
<code>catch</code> skipped; function over	t=300 network fails -> Promise rejects
t=300 network fails -> Promise rejects	t=300 the <code>await</code> throws, right here -> <code>catch</code> runs
-> nobody awaiting inside any try	
-> unhandled rejection	

The `await` keeps the block open across the waiting time. Without it the block is over in microseconds, long before the Promise has anything to say.

The error lives wherever the `await` is. A Promise travels unopened through any number of functions and `try` blocks until something awaits it; that is the `try` whose `catch` runs.

```
function getUser(id: string) {
  try { return fetch(`/users/${id}`); } // no await inside: this catch can NEVER run
  catch (e) { log(e); }               // dead code
}

async function main() {
  try { const res = await getUser("42"); } // the await is inside THIS try
  catch (e) { log(e); }                  // this catch runs on a network failure
}
```

The same rule in every shape. The test is mechanical: is there an `await` on that Promise, inside this `try` block, before the block ends?

Inside a <code>try</code>	Covered?	Why
<code>save(data);</code>	no	never awaited; rejection escapes
<code>await save(data);</code>	yes	thrown at the <code>await</code> , inside the block
<code>return save(data);</code>	no	the Promise leaves the block pending
<code>return await save(data);</code>	yes	waits inside, then returns the value
<code>const p = save(data); other();</code>	no	<code>p</code> never awaited before the block ends
<code>const p = save(data); other(); await p;</code>	yes	awaited before the block ends
<code>items.forEach(async (i) => { await save(i) })</code>	no	the awaits are inside callbacks <code>forEach</code> does not wait for
<code>for (const i of items) await save(i);</code>	yes	each <code>await</code> is inside the block
<code>await Promise.all(items.map(save))</code>	yes	one <code>await</code> covers all of them; first rejection throws

What a rejection does when nobody awaits it. It becomes an *unhandled rejection*: Node prints a warning and, by default in modern versions, crashes the process; a browser logs it to the console. Either way, no `catch` in your code ran, and no cleanup happened.

The fixes, when a `try` encloses no `await`.

Fix	Code	Use when
A: handle here	make the function <code>async</code> ; <code>return await fetch(...)</code> ; after <code>log(e)</code> , <code>throw e</code> or return a fallback	the function should log, add context, retry, or default
B: handle at the caller	delete the <code>try</code> ; keep <code>return fetch(...)</code> ; the caller writes <code>try { await getUser(id) } catch (e) {...}</code>	the function is a pure pass-through

Two things `await` inside a `try` still does not cover. A `fetch` that gets a 404 or 500 fulfils, with `res.ok === false`; the `catch` sees only network failures, so add `if (!res.ok) throw new Error(...)` after the `await`. And a `catch (e)` receives `unknown`; read `e.message` only after `if (e instanceof Error)`.

The review sentence. “The `try` on line *N* encloses no `await`; the Promise leaves the block pending, so the `catch` can never run. Either `async` + `return await`, or drop the `try` and let the caller handle it.”

Say it to yourself for every `try` you meet: “What is awaited inside this block? If nothing, this catch is dead.”

Chaining

“Chaining” in TypeScript means three different things. Recognize each on sight; only the second is new.

1. Method chaining on data. Each `.` takes the result of what is to its left and calls the next method on it. Read left to right, tracing one element through.

```
orders.filter((o) => o.paid).map((o) => o.amount).reduce((a, b) => a + b, 0)
//   keep the paid ones   -> their amounts       -> add them up starting at 0
```

Say: “filter orders to the paid ones, map each to its amount, reduce by adding, starting from 0.” Python: `sum(o.amount for o in orders if o.paid)`. Checks: every step returns a new value (nothing in the chain mutates); the chain’s final value is assigned or returned.

2. Promise chaining with `.then`. The pre-`async/await` way of waiting. Each `.then` is an `await` in disguise; the whole chain is one Promise.

```
fetch(url)
  .then((res) => res.json()) // when fetch settles, parse the body
  .then((data) => data.user) // when that settles, take .user
  .catch((e) => log(e));    // if anything above rejected, log it
```

Say: “fetch; when that settles, parse the body; when that settles, take `.user`; if anything above rejected, log it.” Exact translation:

```
try {
  const res = await fetch(url);
  const data = await res.json();
  return data.user;
} catch (e) { log(e); }
```

Check	Looks like	Problem
Is the chain's result caught?	<code>fetch(url).then(...)</code> alone on a line	fire-and-forget; same as an un-awaited call
Is there a <code>.catch</code> at the end, or is the chain awaited inside a <code>try</code> ?	<code>.then(a).catch(h).then(b)</code>	the <code>.catch</code> covers only what is above it; a rejection in <code>b</code> escapes
Does each <code>.then</code> return?	<code>.then((res) => { res.json() })</code>	braces with no <code>return</code> pass <code>undefined</code> to the next step
Mixed styles	<code>await fetch(url).then((r) => r.json())</code>	fine and common
Mixed styles	<code>const d = fetch(url).then(...)</code> with no <code>await</code>	<code>d</code> holds a Promise
Nested instead of chained	<code>.then((res) => { res.json().then((d) => ...) })</code>	inner Promise not returned; errors inside escape the outer <code>.catch</code>

Each `.then` returns a new Promise, so `.then` after `.then` is sequential: the second waits for the first. `.catch(h)` returns a Promise too, so a chain can continue after it (with `h`’s return value). `.finally(f)` runs `f` either way and passes the result through.

3. Optional chaining `?.` Stops at the first missing link and gives `undefined` instead of throwing.

```
user?.address?.city    // "user, if it exists, dot address, if it exists, dot city"
```

Not related to Promises despite the name. Check: is `undefined` an acceptable result on the next line?

Looks like chaining, is a builder. Fluent query APIs return `this` from every call so the query is assembled step by step. Nothing runs until the final `await`.

```
await knex("users").where({ org }).orderBy("name").limit(20); // runs
knex("users").where({ org }).orderBy("name").limit(20);       // builds a query and drops it;
runs nothing
```

Check: is the finished builder awaited? A builder on its own line with no `await` does no work and no error.

The one rule across all four.

A chain is a single expression with a single final value. Find that value and ask the same question as always: is it assigned, returned, awaited, or passed somewhere? If it is on its own line with nothing catching it, the chain did nothing (data, builder) or did something nobody is listening to (Promise).